

Wstęp do XML – tworzenie interfejsu użytkownika telefonu

1. Czym jest XML?

XML (Extensible Markup Language) to język znaczników używany między innymi w tworzeniu aplikacji na system Android. W Android Studio XML służy głównie do projektowania wyglądu aplikacji, czyli interfejsu użytkownika.

Za pomocą XML możemy tworzyć takie elementy jak:

- przyciski,
- pola tekstowe,
- napisy,
- obrazy,
- układy ekranów,
- formularze.

XML odpowiada za to, jak wygląda ekran aplikacji, natomiast języki takie jak Kotlin lub Java odpowiadają za to, jak działa aplikacja.

2. Budowa pliku XML

Każdy ekran aplikacji może zawierać elementy zapisane w postaci znaczników.

Przykład:

```
<TextView  
    android:text="Witaj!"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content" />
```

Komentarze w XML

Podobnie jak w innych językach, w XML można zostawiać sobie notatki, które nie wpływają na działanie kodu:

```
<!-- To jest komentarz, np. przypomnienie co robi ten element -->
```

Przydaje się to, gdy layout robi się dłuższy i trudniej się w nim połapać.

3. Podstawowe wartości rozmiaru

W XML bardzo często używamy dwóch wartości:

`wrap_content`

Element zajmuje tylko tyle miejsca, ile potrzebuje jego zawartość.

```
android:layout_width="wrap_content"
```

Jeżeli napis ma 5 znaków, element będzie miał szerokość potrzebną tylko dla tych znaków.

`match_parent`

Element zajmuje całą dostępną przestrzeń rodzica.

```
android:layout_width="match_parent"
```

Przycisk lub układ może wtedy zajmować całą szerokość ekranu.

4. Layout – układ ekranu

Elementy aplikacji muszą znajdować się wewnątrz odpowiedniego układu.

Jednym z prostych układów jest `LinearLayout`.

`LinearLayout` pozwala układać elementy:

- pionowo,

- poziomo.

Układ pionowy:

```
android:orientation="vertical"
```

Elementy będą pojawiały się jeden pod drugim.

Układ poziomy:

```
android:orientation="horizontal"
```

Elementy będą pojawiały się obok siebie.

```
<LinearLayout
```

```
    xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
```

```
    android:orientation="vertical">
```

```
</LinearLayout>
```

5. Tworzenie napisu – TextView

Do wyświetlania tekstu używamy elementu `TextView`.

```
<TextView
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="Witaj w aplikacji!"
```

```
    android:textSize="24sp" />
```

Najważniejsze właściwości:

- `android:text` – wyświetlany tekst,
- `android:textSize` – rozmiar tekstu,
- `android:textColor` – kolor tekstu.

Kolor tła – `background`

Tak jak można pokolorować tekst, można też pokolorować tło samego elementu:

```
<TextView
```

```
    android:layout_width="match_parent"

    android:layout_height="wrap_content"

    android:text="Witaj!"

    android:background="#FFEB3B"

    android:textColor="#000000"

    android:padding="16dp" />
```

`android:background` działa na każdym elemencie (`TextView`, `Button`, `EditText`, a nawet całym `LinearLayout`) – nie tylko na tekście.

6. Tworzenie przycisku – `Button`

Przycisk tworzymy za pomocą znacznika `Button`.

```
<Button
```

```
    android:layout_width="wrap_content"

    android:layout_height="wrap_content"

    android:text="Kliknij mnie" />
```

W tym przypadku przycisk zajmie całą szerokość ekranu.

7. Pole do wpisywania tekstu – EditText

Jeżeli użytkownik ma wpisać tekst, używamy `EditText`.

```
<EditText  
  
    android:layout_width="match_parent"  
  
    android:layout_height="wrap_content"  
  
    android:hint="Wpisz swoje imię" />
```

`android:hint` wyświetla podpowiedź wewnątrz pola. Po kliknięciu użytkownik może wpisać własny tekst, a podpowiedź znika.

8. Identyfikator elementu – `android:id`

Do tej pory nasze elementy nie miały żadnej "nazwy własnej" – były tylko ładnym, ale nieruchomym obrazkiem. Żeby aplikacja zaczęła działać (np. żeby przycisk reagował na kliknięcie), trzeba nadać elementowi identyfikator:

```
<Button  
  
    android:id="@+id/buttonLogin"  
  
    android:layout_width="match_parent"  
  
    android:layout_height="wrap_content"  
  
    android:text="Zaloguj się" />
```

- Znak `+` mówi Androidowi: "stwórz ten identyfikator, jeśli jeszcze nie istnieje".
- Dzięki `android:id` będzie można później, w kodzie Javy, "znaleźć" ten konkretny przycisk poleceniem `findViewById()` i np. dopisać mu reakcję na kliknięcie.

To jest najważniejszy element łączący XML (wygląd) z kodem (działanie) – bez `id` żaden przycisk "nie zareaguje" na dotknięcie, bo program nie będzie wiedział, o który element chodzi.

9. Tworzenie prostego interfejsu aplikacji

Poniżej znajduje się przykład prostego ekranu logowania – to jest nasz główny przykład, do którego odnosi się legenda na końcu dokumentu.

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<LinearLayout
```

```
    xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
```

```
    android:orientation="vertical"
```

```
    android:padding="20dp">
```

```
    <TextView
```

```
        android:id="@+id/textViewHeader"
```

```
        android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"
```

```
        android:text="Logowanie"
```

```
        android:textSize="28sp" />
```

```
    <EditText
```

```
        android:id="@+id/editTextLogin"
```

```
        android:layout_width="match_parent"
```

```
        android:layout_height="wrap_content"
```

```
        android:hint="Login" />
```

```
    <EditText
```

```
        android:id="@+id/editTextPassword"
```

```
android:layout_width="match_parent"

android:layout_height="wrap_content"

android:hint="Hasło" />
```

```
<Button
```

```
    android:id="@+id/buttonLogin"

    android:layout_width="match_parent"

    android:layout_height="wrap_content"

    android:text="Zaloguj się"

    android:layout_marginTop="20dp" />
```

```
</LinearLayout>
```

Ten kod tworzy ekran zawierający:

1. Nagłówek "Logowanie".
2. Pole do wpisania loginu.
3. Pole do wpisania hasła.
4. Przycisk logowania.

10. Odstępy i marginesy

W Androidzie możemy ustawiać odstępy między elementami.

Margin

Margin to odstęp na zewnątrz elementu.

```
android:layout_marginTop="20dp"
```

Przykład:

```
<Button
```

```
android:layout_width="match_parent"

android:layout_height="wrap_content"

android:text="Zaloguj się"

android:layout_marginTop="20dp" />
```

Przycisk będzie oddalony od elementu znajdującego się nad nim.

11. Jednostki używane w Android XML

W Androidzie najczęściej używamy:

dp (density-independent pixels)

Jednostka używana do określania rozmiarów elementów, marginesów i paddingu. Wygląda tak samo niezależnie od tego, jak "gęsty" jest ekran telefonu.

```
android:padding="16dp"
```

sp (scale-independent pixels)

Używamy głównie do rozmiaru tekstu. Różni się od **dp** tym, że uwzględnia też ustawienia dostępności telefonu – jeśli użytkownik zwiększy w telefonie rozmiar czcionki systemowej, tekst w **sp** też się powiększy.

```
android:textSize="20sp"
```

Podsumowanie

XML w Android Studio służy do tworzenia wyglądu aplikacji.

Najważniejsze elementy:

Element	Zastosowanie
---------	--------------

LinearLayout Układ elementów

TextView Wyświetlanie tekstu

EditText Wprowadzanie tekstu

Button Przycisk

ImageView Wyświetlanie obrazów

Najważniejsze właściwości:

- `android:layout_width`
- `android:layout_height`
- `android:text`
- `android:textSize`
- `android:textColor`
- `android:background`
- `android:padding`
- `android:layout_margin`
- `android:hint`
- `android:id`

Dzięki XML możemy zaprojektować cały wygląd ekranu telefonu bez tworzenia jego działania w kodzie Kotlin lub Java. W kolejnej lekcji połączymy ten wygląd z kodem, żeby przycisk faktycznie coś robił po kliknięciu.

LEGENDA

Nagłówek pliku

Kod	Co robi
<code><?xml version="1.0" encoding="utf-8"?></code>	Deklaracja pliku XML – wersja standardu i kodowanie znaków (utf-8 obsługuje polskie znaki: ą, ę, ł, ż itd.)

Element główny – `<LinearLayout>`

Kod	Co robi
<code><LinearLayout ...></code>	Kontener, w którym mieszczą się wszystkie pozostałe elementy ekranu
<code>xmlns:android="..."</code>	Namespace – pozwala używać atrybutów <code>android:...</code> w tym pliku
<code>android:layout_width="match_parent"</code>	Kontener zajmuje całą szerokość ekranu
<code>android:layout_height="match_parent"</code>	Kontener zajmuje całą wysokość ekranu
<code>android:orientation="vertical"</code>	Elementy w środku układają się jeden pod drugim
<code>android:padding="20dp"</code>	Cała zawartość kontenera jest odsunięta o 20dp od jego krawędzi ze wszystkich stron

Element 1 – <TextView> (nagłówek)

Kod	Co robi
<code>android:id="@+id/textViewHeader"</code>	Nadaje elementowi unikalną nazwę, po której można się do niego odwołać w kodzie
<code>android:layout_width="wrap_content"</code>	Szerokość dopasowana do długości napisu "Logowanie"
<code>android:layout_height="wrap_content"</code>	Wysokość dopasowana do rozmiaru tekstu
<code>android:text="Logowanie"</code>	Tekst wyświetlany na ekranie
<code>android:textSize="28sp"</code>	Rozmiar czcionki nagłówka (duży, bo to tytuł ekranu)

Element 2 – <EditText> (login)

Kod	Co robi
<code>android:id="@+id/editTextLogin"</code>	Identyfikator pola loginu – potrzebny, żeby w kodzie odczytać, co użytkownik wpisał
<code>android:layout_width="match_parent"</code>	Pole rozciąga się na całą szerokość ekranu (minus padding kontenera)
<code>android:layout_height="wrap_content"</code>	Standardowa wysokość pola tekstowego

`android:hint="Login"`

Szary podpowiadający tekst widoczny, dopóki pole jest puste

Element 3 – `<EditText>` (hasło)

Kod

Co robi

`android:id="@+id/editTextPassword"`

Identyfikator pola hasła – osobny od pola loginu, żeby móc je rozróżnić w kodzie

`android:layout_width="match_parent"`

Tak jak wyżej – pełna szerokość ekranu

`android:layout_height="wrap_content"`

Standardowa wysokość

`android:hint="Hasło"`

Podpowieź informująca, że to pole na hasło

(Uwaga dla uczniów: w tym podstawowym przykładzie wpisane hasło byłoby widoczne "jawnie" jako zwykły tekst. Ukrywanie hasła kropkami to dodatkowy atrybut `android:inputType="textPassword"` – można to pokazać jako ciekawostkę lub zadanie dodatkowe.)

Element 4 – `<Button>` (przycisk logowania)

Kod

Co robi

`android:id="@+id/buttonLogin"`

Identyfikator przycisku – to on pozwoli w kolejnej lekcji "złapać" kliknięcie w kodzie

```
android:layout_width="match_
parent"
```

Przycisk zajmuje całą szerokość ekranu

```
android:layout_height="wrap_
content"
```

Standardowa wysokość przycisku

```
android:text="Zaloguj się"
```

Napis widoczny na przycisku

```
android:layout_marginTop="20
dp"
```

Odstęp 20dp od elementu znajdującego się nad nim (pola hasła) – żeby przycisk nie "sklejał się" wizualnie z resztą

Zamknięcie

Kod

Co robi

```
</LinearLayo
ut>
```

Zamyka kontener – każdy element otwierający musi mieć swój odpowiednik zamykający